

Meeting 02-04

Inspecting NLTK Treebank Corpus

Lars Heijnen

Example Parse Tree (Raw Format)

Raw string format (*treebank.raw()*)

- Syntactic structure using nested parentheses.
- Each line starts with a phrase type (e.g., NP, VP, S).
- Some labels include *function tags*, like -*SBJ* (subject).
 - *Function tags* explain what a phrase does in the sentence.

```
( (S
  (NP-SBJ
    (NP (NNP Pierre) (NNP Vinken) )
    ( , , )
  (ADJP
    (NP (CD 61) (NNS years) )
    (JJ old) )
    ( , , )
  (VP (MD will)
    (VP (VB join)
      (NP (DT the) (NN board) )
      (PP-CLR (IN as)
        (NP (DT a) (JJ nonexecutive) (NN director) ))
      (NP-TMP (NNP Nov.) (CD 29) )))
    ( . . ) ))
  ( (S
    (NP-SBJ (NNP Mr.) (NNP Vinken) )
    (VP (VBZ is)
      (NP-PRD
        (NP (NN chairman) )
        (PP (IN of)
          (NP
            (NP-SBJ (NNP Pierre) (NNP Vinken)) ← Subject noun phrase
```

Example POS Tagging Format

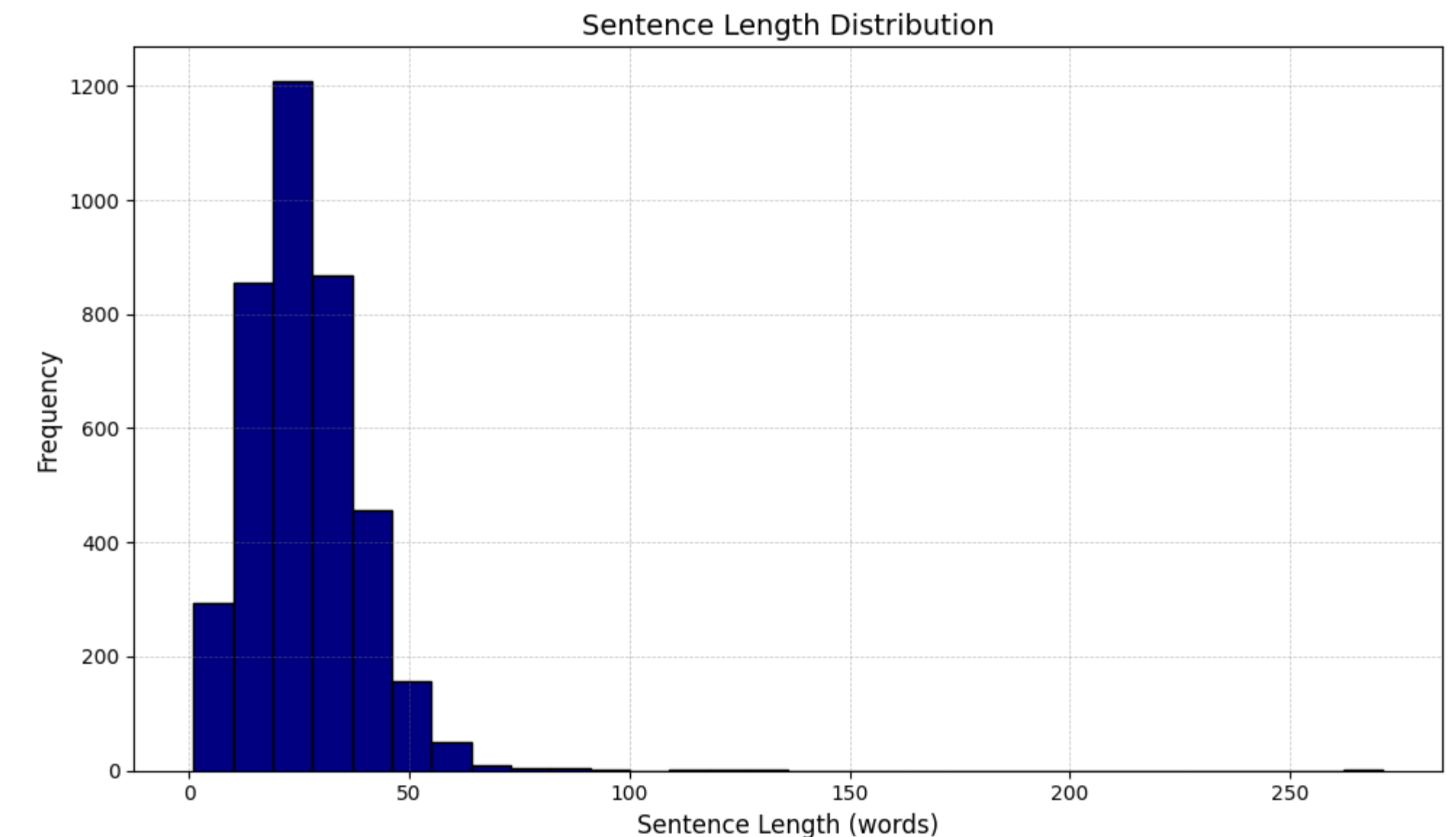
POS Tagging Format (*treebank.tagged_sents()*)

```
[('Pierre', 'NNP'), ('Vinken', 'NNP'), (',', ','), ('61', 'CD'), ....
```

- Returns tokenized sentences with POS tags
- Useful for linguistic feature extraction and tagging tasks.

Stats on NLTK Treebank Corpus

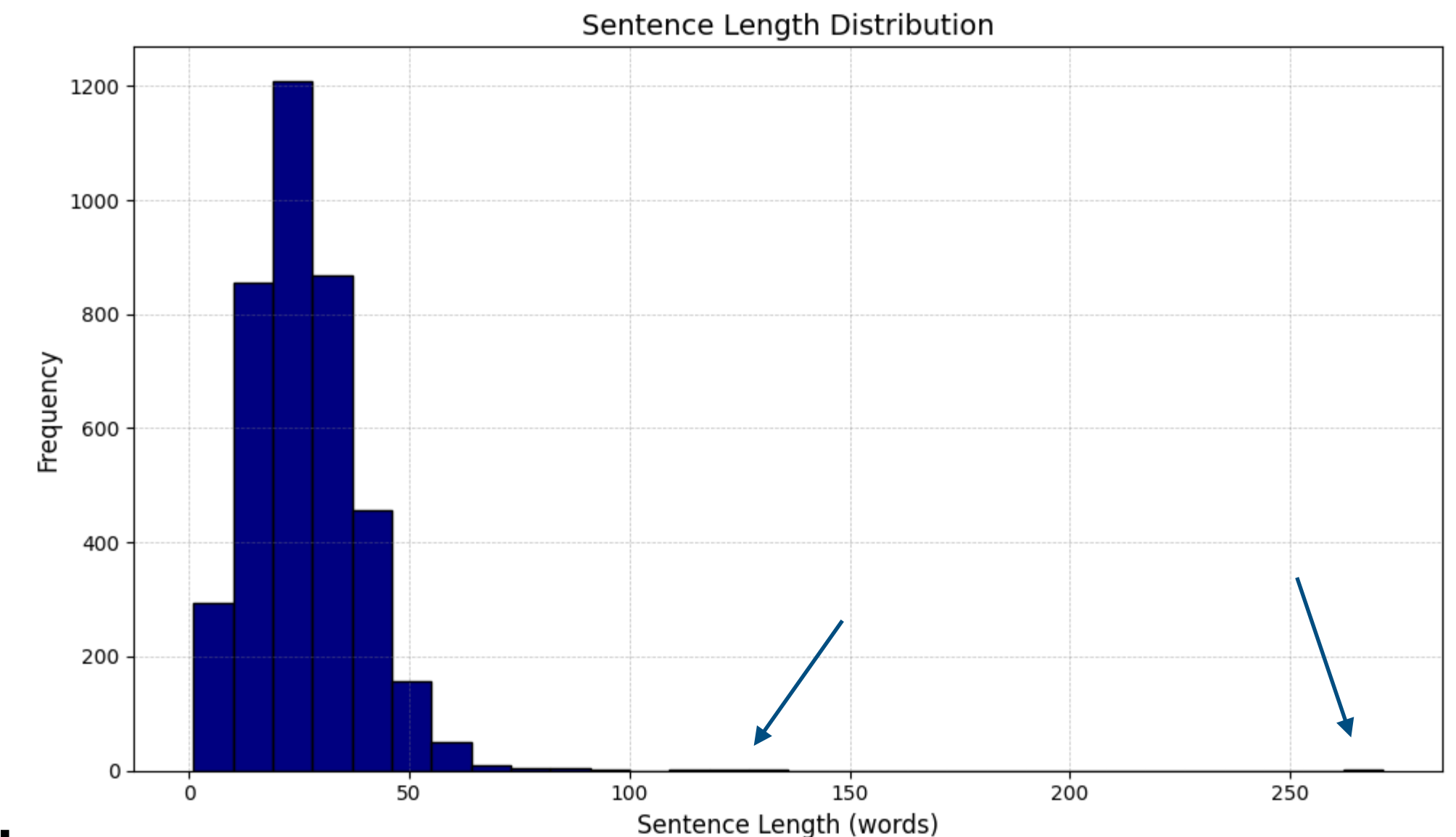
- Total number of sentences: **3914**
- Total number of tokens: **100676**
- Total number of unique tokens: **12408**
- Average sentence length: **25.722**
- Variance of sentence lengths: **165.557**
- Standard deviation of sentence lengths: **12.869**
- Number of unique POS tags: **46**



Stats on NLTK Treebank Corpus

Spread, Mean, Standard Deviation and Variance

- Typical sentence length tends to deviate from the mean by around **13 words (≈ 12.869)**.
- A small portion of sentences is much longer:
 - Stretches the tail.
 - Contributes to variance.
- While most sentences cluster near the **25.722** average, there is significant variability.



Variance of sentence length ≈ 165.56
Standard deviation of sentence length ≈ 12.87

More Stats on NLTK Treebank Corpus

Top 10 Frequent Tokens

- **Punctution** shows up since they are treated as separate **tokens**.
- **Trace Tokens** represent *syntactic movement* or *empty categories*:
 - They are **not** real words, but placeholders to maintain *syntactic structure*.

Token	Count
,	4885
the	4045
.	3828
of	2319
to	2164
a	1878
in	1572
and	1511
*-1	1123
0	1099

Token	Meaning
*	Generic placeholder
*-1	Traced of moved constituent
T-1	“T” for Wh-trace (e.g. in questions)
0	Empty subject

Think of it like this

Language is messy

- People say things like:
 - What did Jogn say he wanted to buy?
- In surface form, “what” is at the beginning. But syntactically, it’s the object of “buy”; it moved from its logical place at the end.
- To **encode that movement**, Treebank adds.
 - ***T*-1** where “What” originally was,
 - ***-1** to mark where it got moved to.
- This allows a machine (or linguist) to know:
 - What moved,
 - Where it came from,
 - Where it ended up.

Token	Count
,	4885
the	4045
.	3828
of	2319
to	2164
a	1878
in	1572
and	1511
*-1	1123
0	1099

Token	Meaning
*	Generic placeholder
*-1	Traced of moved constituent
T-1	“T” for Wh-trace (e.g. in questions)
0	Empty subject

TL;DR

Trace tokens help preserve “who did what to whom” in sentences where words are out of order.

Goal: Predict sentence length using BERT sentence embeddings

But what to do with trace tokens?

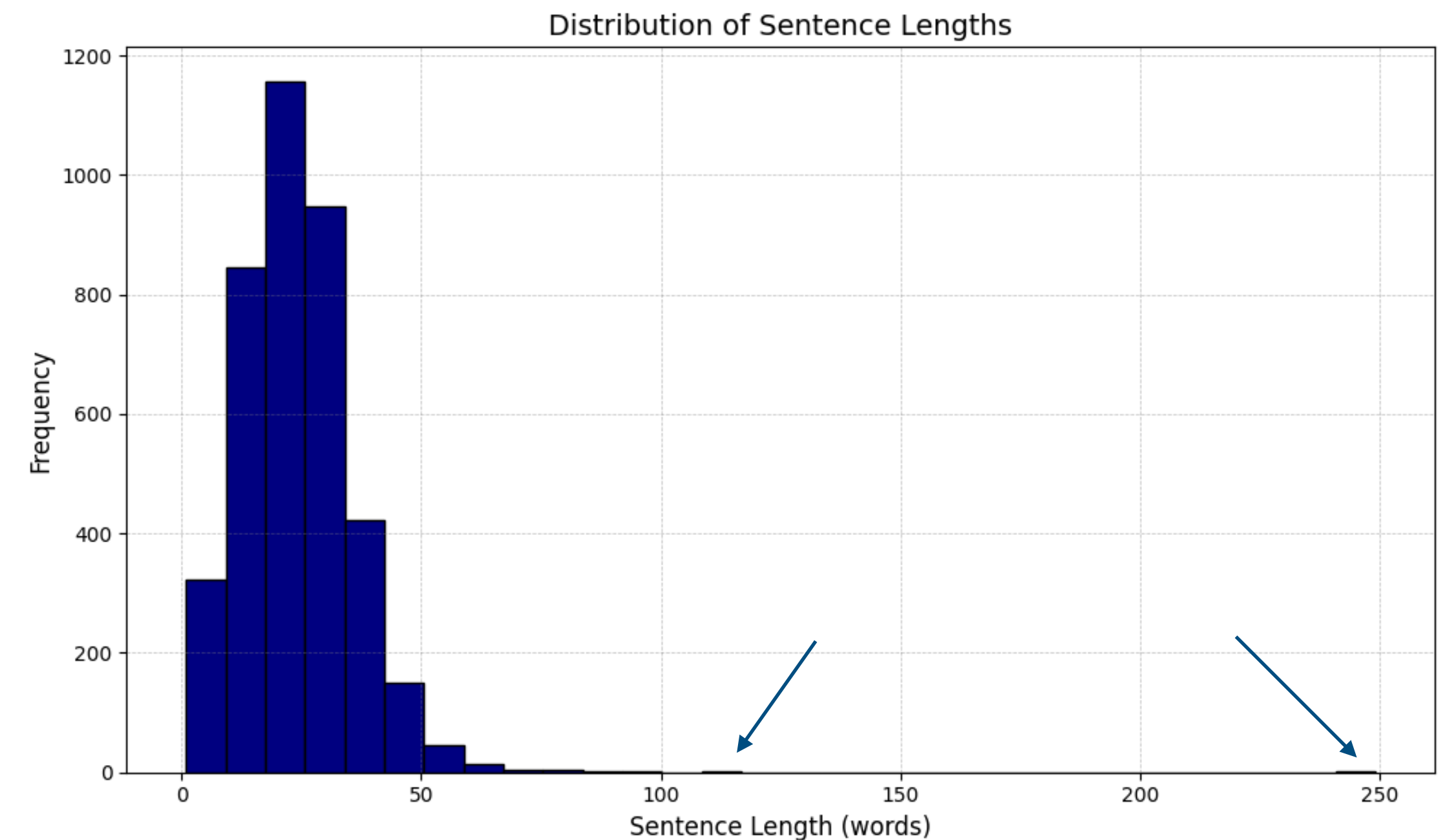
Do not include them. Why?

- Not actual lexical content:
 - They're invisible in real language use.
- Semantically empty:
 - They don't carry meaning that BERT can interpret in context.
- Bias sentence length:
 - Including trace tokens in the word count makes length predictions less accurate.

Use clean sentences that resemble natural input.

Stats on Cleaned NLTK Treebank Corpus

- Total number of sentences: **3914**
- Total number of tokens: **94084**
- Total number of unique tokens: **11968**
- Average sentence length: **24.038**
- Variance of sentence lengths: **140.696**
- Standard deviation of sentence lengths: **11.863**
- Number of unique POS tags: **45** (removal of '-NONE-')



More Stats on Cleaned NLTK Treebank Corpus

Top 10 Frequent Tokens

- **Punctution** shows up since they are treated as separate tokens.
- **Trace Tokens** are gone.

Token	Count
,	4885
the	4045
.	3828
of	2319
to	2164
a	1878
in	1572
and	1511
's	864
for	817

Comparison with Original Data

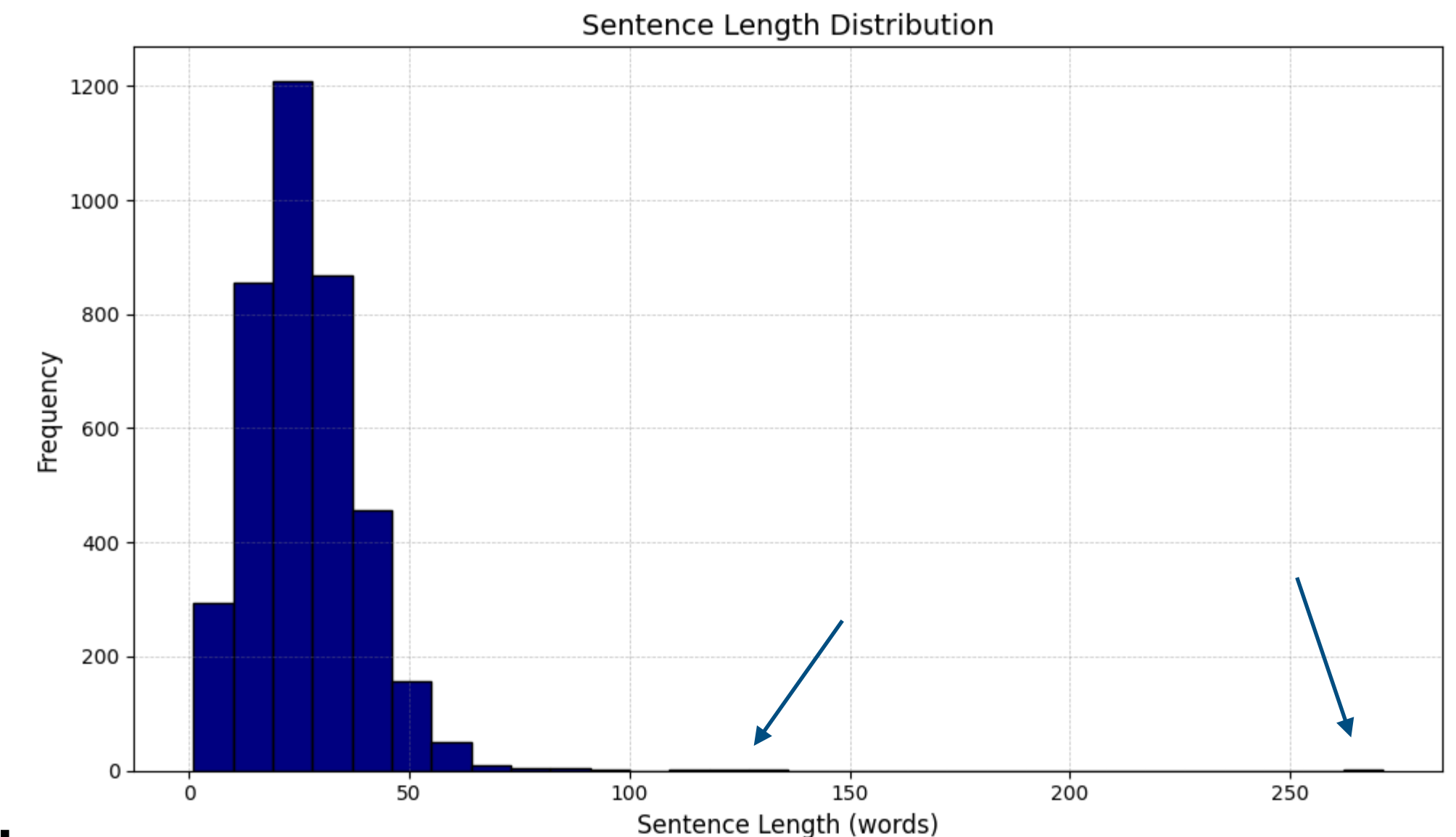
- Sentences removed: **0**
- Words removed: **6592**
- Unique words removed: **440**
- Change in average sentence length: **-1.684**
- Change in sentence length variance: **-24.861**
- POS tags removed: **1**

Token	Count
,	4885
the	4045
.	3828
of	2319
to	2164
a	1878
in	1572
and	1511
's	864
for	817

Stats on NLTK Treebank Corpus

Spread, Mean, Standard Deviation and Variance

- Typical sentence length tends to deviate from the mean by around **12 words (≈ 11.863)**.
- But still, a small portion of sentences is much longer:
 - Stretches the tail.
 - Contributes to variance.
- While most sentences cluster near the **24.038** average, there is significant variability.



Variance of sentence length ≈ 165.56
Standard deviation of sentence length ≈ 12.87

What next?

- Deal with **outliers**:
 - How many are there?
 - Data errors? Or legitimate?
 - Experiment with log transform:
 - Helps with skewed distribution.

Feedback

1. Outlier Rejection Using Quartiles:

The note suggests using a standard method to remove outliers based on quartiles. Typically, you first calculate the interquartile range (IQR), which is the difference between the third quartile (Q3) and the first quartile (Q1). Although the note mentions both “2x interquartile distance” and “1.5x greater than Q3 or Q1,” the common approach is to remove any data points that fall below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$. There are standard functions available to compute these quartiles and the IQR, and a boxplot can be used to visualize this: the “box” represents Q1, Q2 (median), and Q3, while the “whiskers” typically extend to the most extreme data points within $1.5 \times IQR$ of the quartiles.

2. Including POS Tags in Text Analysis:

The note also mentions incorporating Part-of-Speech (POS) tags when counting tokens. This means that when processing text, you should also record and analyze the POS tags (like nouns, verbs, adjectives, etc.) associated with each token. Performing frequency counts on these POS tags can provide insights into the text’s structure and usage patterns.

In **summary**, the note outlines two tasks:

Data Cleaning: Remove outliers using a quartile-based method (commonly using a $1.5 \times IQR$ threshold).

Text Analysis: Include POS tags in token counts to perform frequency analysis on these tags.

Recap of Last Session (03-04)

Main focus

Exploring the **NLTK Treebank Corpus** to analyze sentence structures.

Key Insights

- **Treebank Format:** Parse trees include function tags (e.g., -SBJ) and trace tokens (*, *-1, *T*-1, 0) to capture syntactic movement.
- **Trace Tokens:** Useful for linguistics but **semantically empty** → **excluded** from BERT input to avoid bias in sentence length prediction.

Recap of Last Session (03-04)

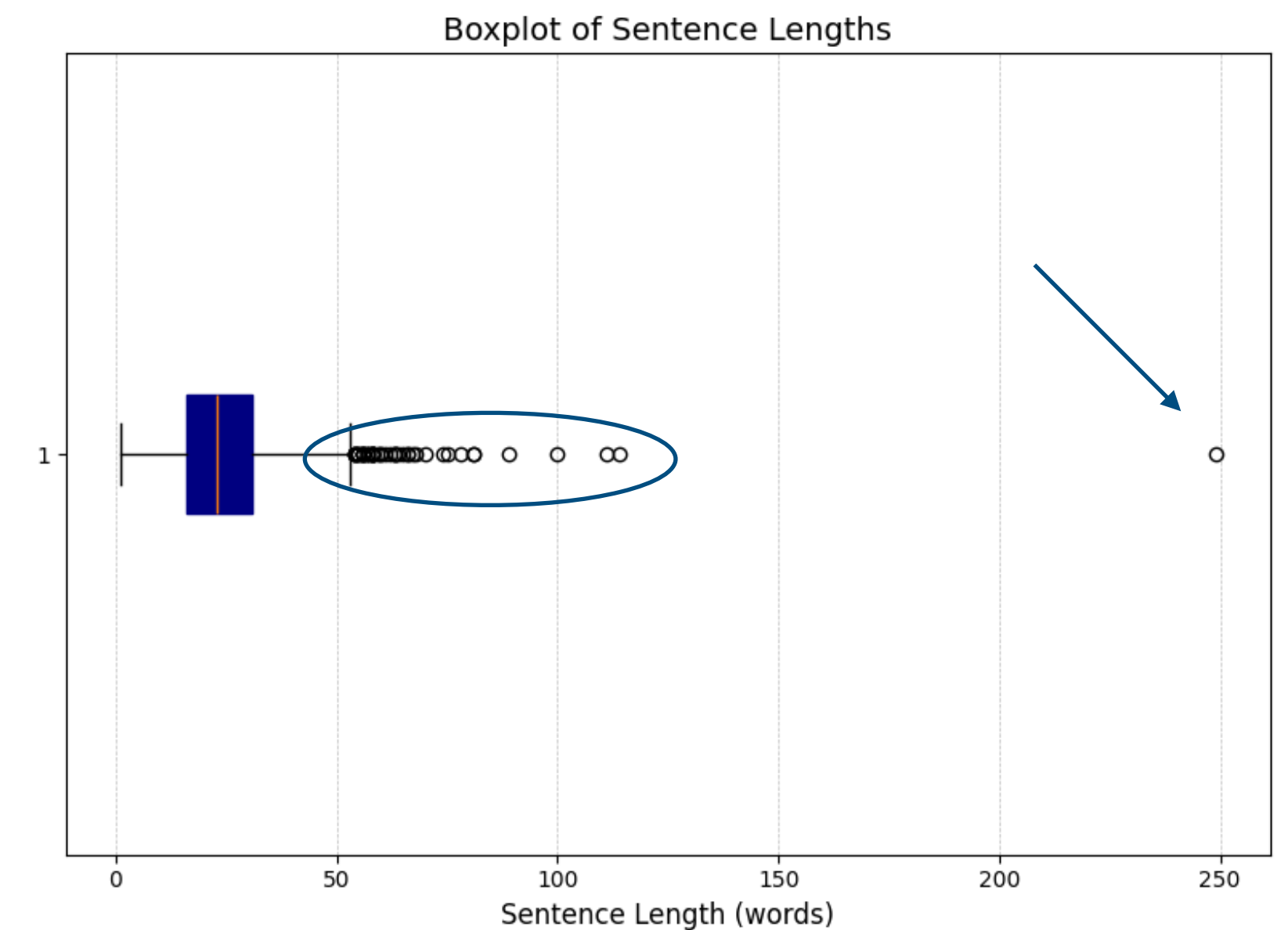
Corpus Statistics

- Raw Corpus: 3914 sentences, avg. length = 25.72, std. dev. = 12.87.
- Cleaned Corpus (no trace tokens): avg. length = 24.04, std. dev. = 11.86.
- Tokens removed: 6592 (trace tokens & placeholders).
- POS tags reduced from 46 → 45 (removal of -NONE-).

Outlier Stats on NLTK Treebank Corpus

Outliers

- Boxplot shows what we have seen earlier:
 - Most sentences are between ~10 to ~50 words.
 - Right-skewed distribution.
 - These outliers **likely** cause the high variance.



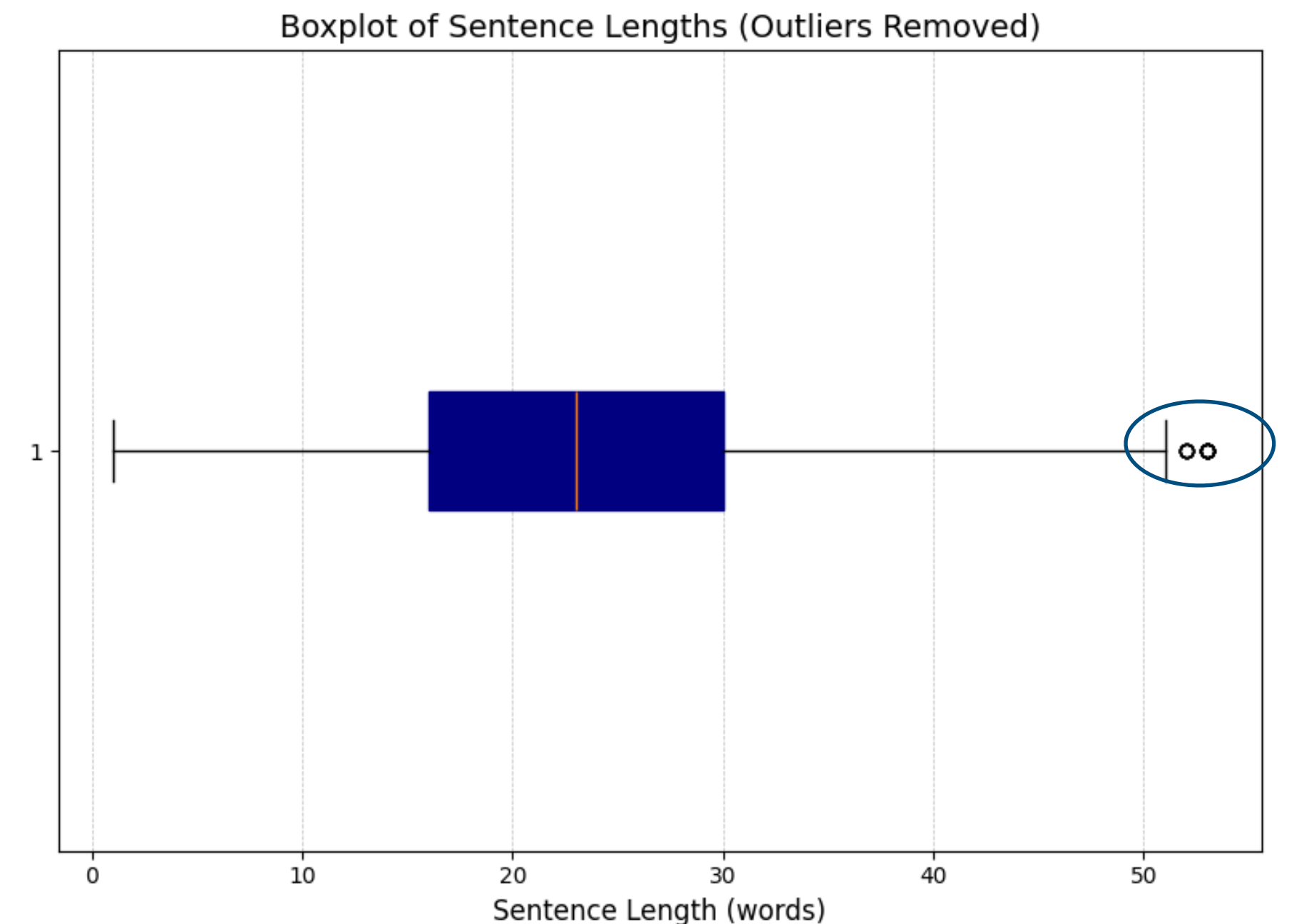
Extreme outlier at ~250 words

New Stats on NLTK Treebank Corpus

Removing outliers

Apply IQR Method:

- Calculate sentence lengths for all cleaned sentences
- Compute 1st (Q1) and 3rd (Q3) quartiles of sentence lengths
- Determine interquartile range ($IQR = Q3 - Q1$)
- Define lower/upper bounds: $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$
- Remove sentences with lengths outside these bounds (outliers)

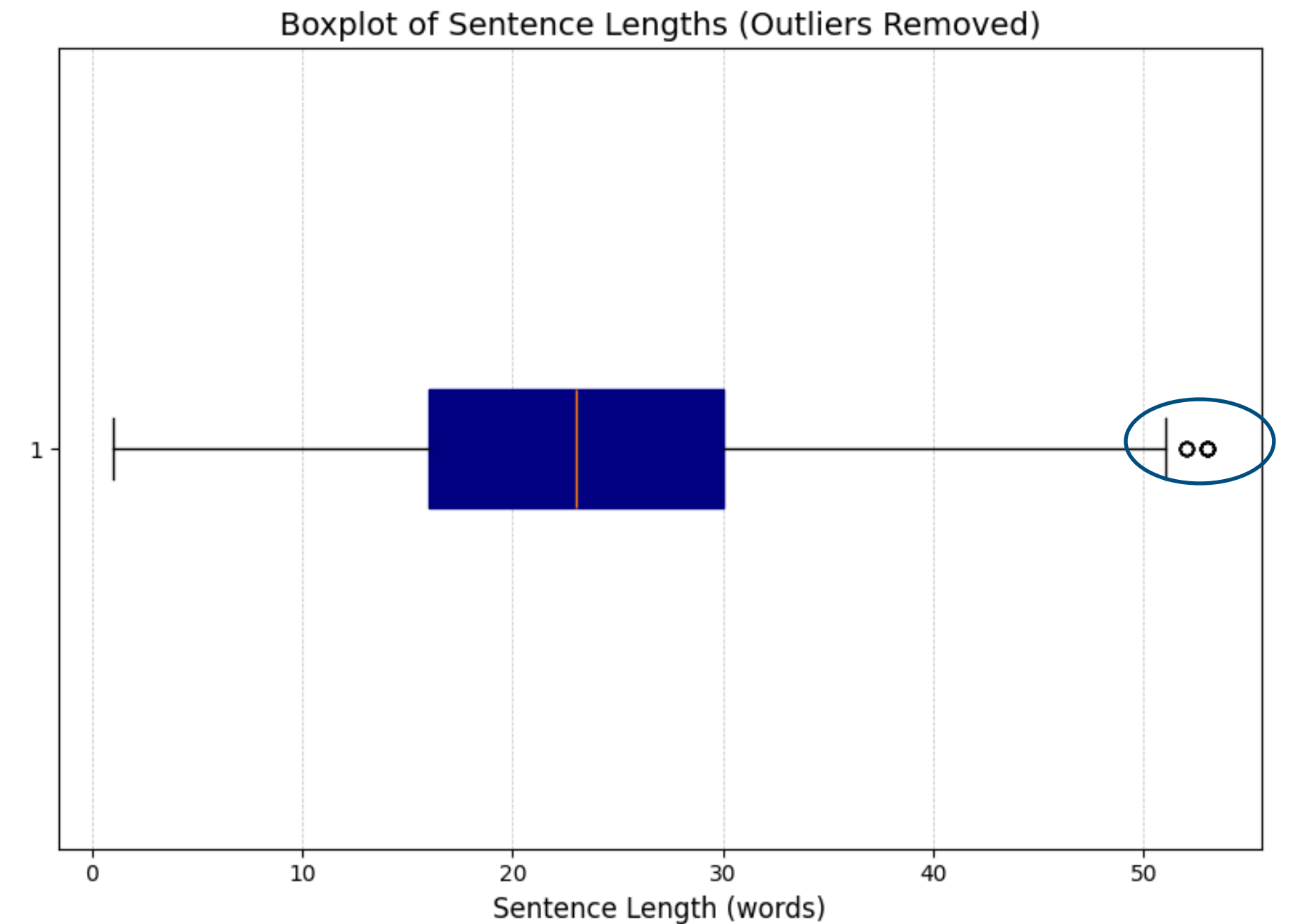


Marked sentences are just outside the outlier threshold

New Stats on NLTK Treebank Corpus

Outliers removed, can we measure that?

- Brings sentence count down from 3914 to 3868.
 - That's only **1.18%** of total amount of sentences.
- **Variance** is down to 106.825, from 140.696.
 - 31.7% reduction.
- **Standard Deviation** is down to 10.337, from 11.863.
 - 14.8% reduction.

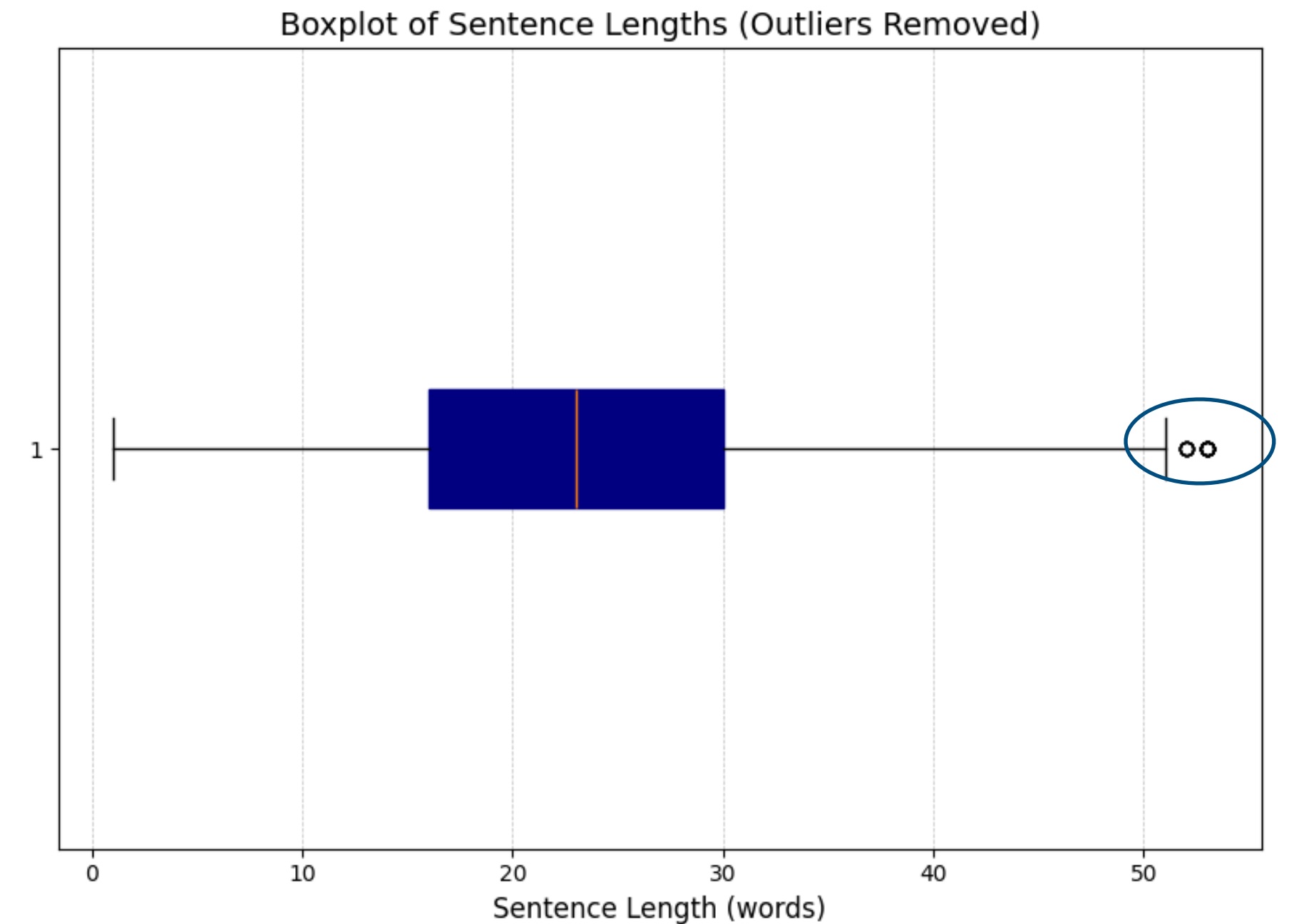


Marked sentences are just outside the outlier threshold

New Stats on NLTK Treebank Corpus

What does this imply?

- The removed sentences were **statistical outliers**, which skew the spread of the data.
- **Variance** and **standard deviation** are sensitive to **extreme values**, so even a **small number of extreme cases** can heavily impact the overall dispersion.
- This shows that the outliers were disproportionately contributing to variability in sentence lengths.

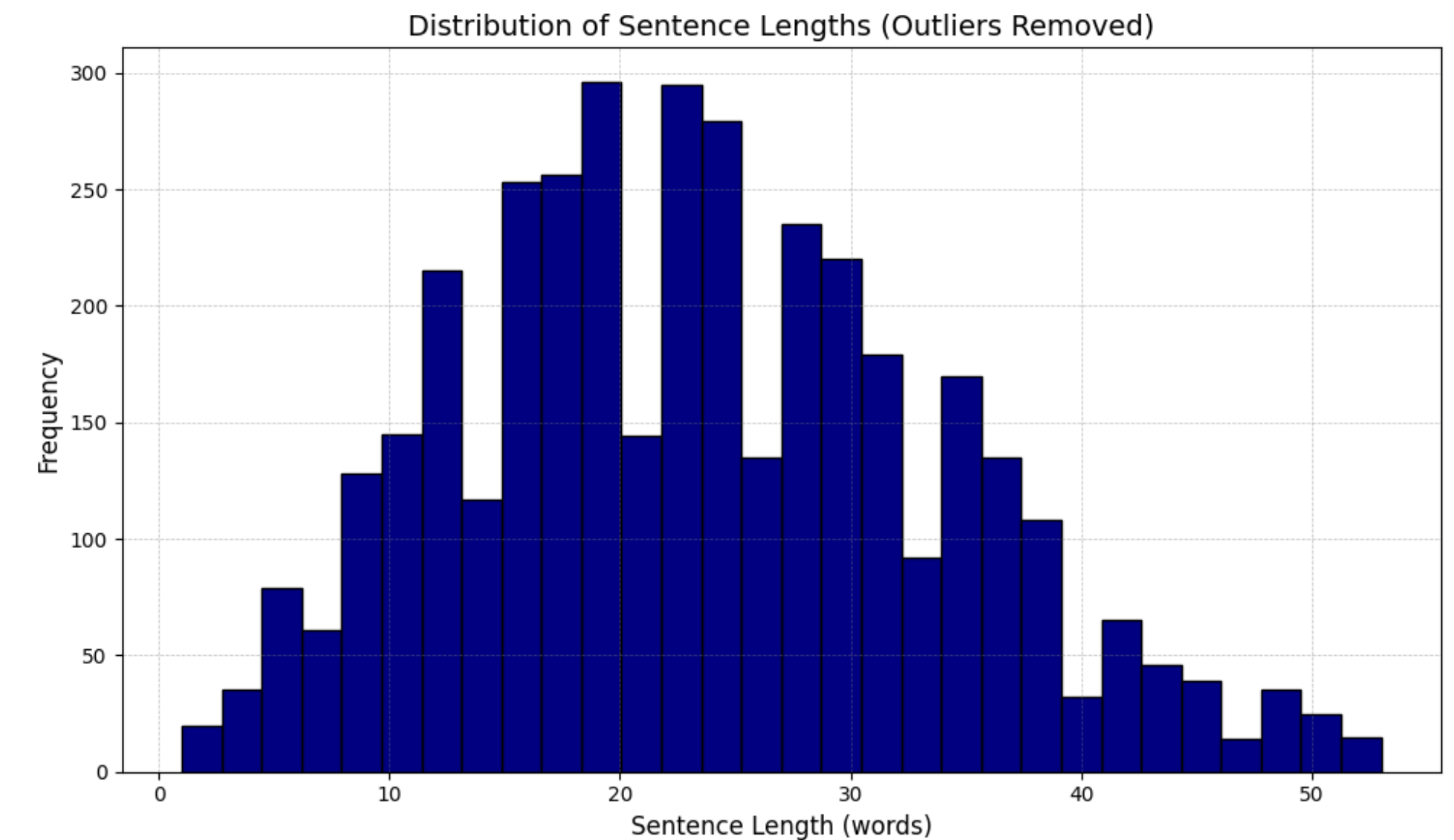


Marked sentences are just inside the outlier threshold

New Stats on NLTK Treebank Corpus

Updated distribution

- Less skewed.
- Most sentences fall between ~10 and ~40 words.
 - Peak is visible around 16–25 words.
- This version better reflects the “typical” sentence in the dataset.



Looks better!

BERT

Context matters

Key idea: The embedding of a word should change based on the sentence it is in.

How?

- BERT uses transformer architecture, in specific, *self-attention*.
- Self-attention lets the model weigh all words in a sequence to encode each word, capturing long-range dependencies.

BERT

Let's trace how BERT generates embeddings

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Tokenization: Done using WordPiece

- WordPiece break down words into sub-word units.
- BERT adds special tokens:
 - [CLS]:
 - Prepend to the beginning of every input sentence.
 - The final hidden state aggregates the representation of the entire sentence.

BERT

Let's trace how BERT generates embeddings, continued

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Tokenization: Done using WordPiece

- BERT adds special tokens:
 - [SEP]:
 - Appended to the *end* of each sentence.
 - Used to indicate the end of a sentence.

BERT

Let's trace how BERT generates embeddings, continued

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Tokenization: Done using WordPiece

- BERT adds special tokens:
 - [PAD]:
 - Used to make all sentences in the batch the same length
 - I've set it to false, meaning it processes each sentence individually.

BERT

Let's trace how BERT generates embeddings, continued

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Wrapping up: The tokenizer converts the input text into a sequence of Token IDs.

- Integers representing each token.

BERT

Input embeddings

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Token Embeddings: Learned embeddings for each token ID in the vocabulary.

BERT

How does the embedding process work?

```
inputs = tokenizer(text, padding=False, truncation=True, return_tensors='pt').to(device)
```

- No extra padding tokens are added (only one sentence at a time).
- If a sentence is too long, it is shortened —> no issue.
- The output is a PyTorch tensor.

BERT

How does the embedding process work?

```
model = BertModel.from_pretrained('bert-base-uncased')
```

- The model takes the token IDs, and computes contextualized embeddings for each token.
- It outputs a tensor of shape $(1, seq_len, 768)$:
 - 1 : batch size (one sentence)
 - seq_len : number of tokens (including [CLS] and [SEP])
 - 768 : hidden size of BERT-base

BERT

Extracting the [CLS] embedding

```
emb = outputs.last_hidden_state[:, 0, :].cpu().numpy()  
embeddings.append(emb[0])
```

- [CLS] captures the sentence's overall meaning, encoding word order, syntax, and semantics.
- `[0, 0, 0]` selects the first token (the [CLS] token) for the batch.
- The result is a vector of length 768 (for BERT-base).
- This vector is appended to the list of embeddings.

BERT

Output

- After processing all sentences, a NumPy array is returned (*num_sentences*, 768).
- The vector is the embedding of the [CLS] token from BERT, representing the entire sentence.

BERT

Summarizing

Step	Details
Input	Cleaned, filtered sentences (strings)
Tokenization	Tokenization, adds [CLS] and [SEP]
Encoding	Converts tokens to IDs, creates PyTorch tensors
BERT Forward	Computes contextualized embeddings for all tokens
Extract [CLS]	Takes the first token's embedding (shape: 768)
Output	NumPy array of shape (num_sentences, 768)
Represents	The overall meaning/context of each sentence, as learned by BERT

Employing Linear Regression

What does it take as input?

- Features (`X_sentence_features`):
 - This is a NumPy array of shape `(num_sentences, 768)`.
 - Each row is a BERT [CLS] embedding for a cleaned, filtered sentence.
 - **Input to the model:** Each sentence is represented as a 768-dimensional vector.
- Target (`y`):
 - This is a list of integers.
 - Each value is the **length of the corresponding sentence** (i.e., the number of words in that sentence).
 - **Target for the model:** The actual sentence length.

Employing Linear Regression

How does it work?

Train-test split:

- The data is split into training (80%) and test (20%) sets.
- $X_{\text{train}}/y_{\text{train}}$ are used to train the model.
- $X_{\text{test}}/y_{\text{test}}$ are used to evaluate the model.

Model training:

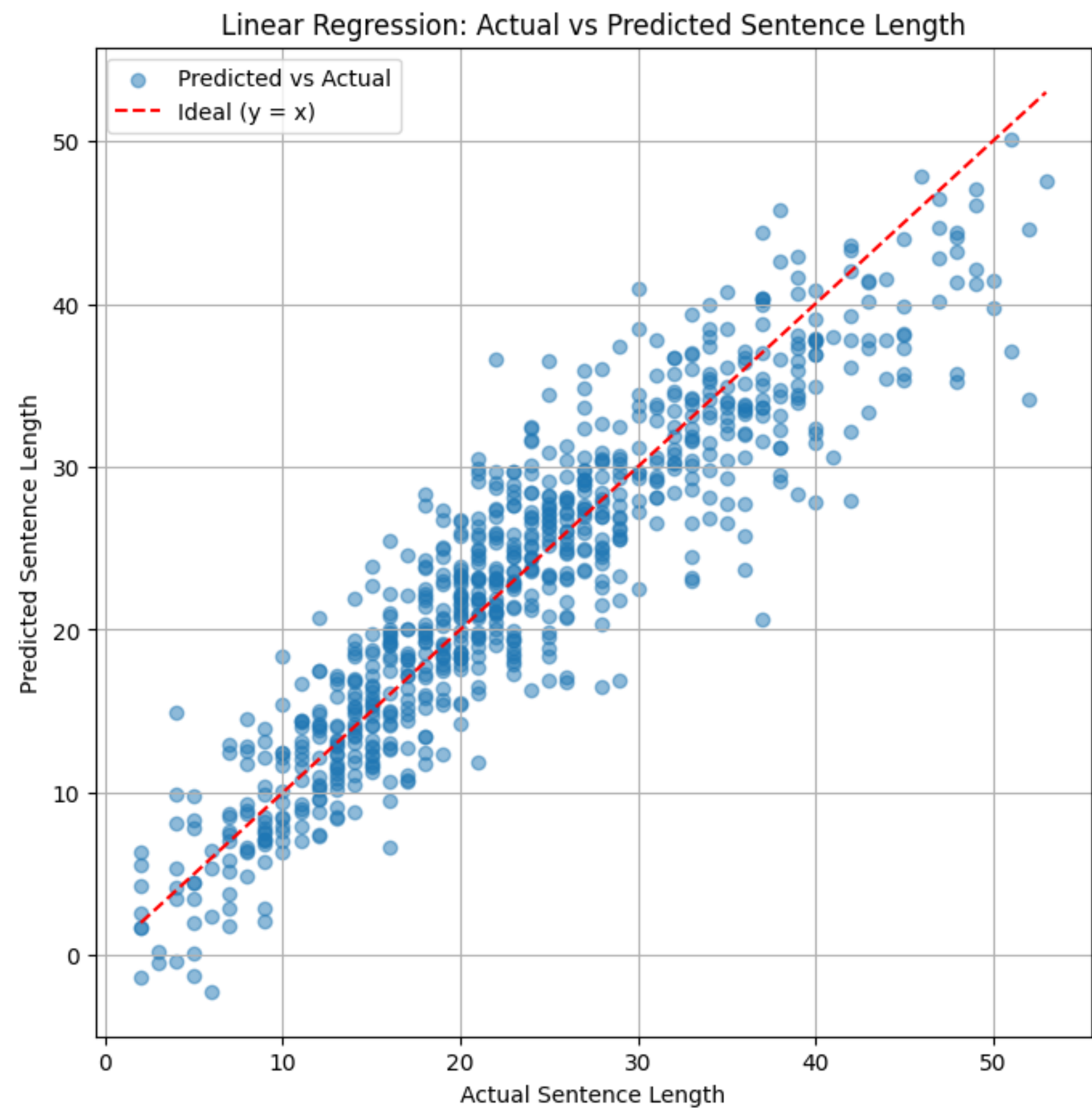
- A LinearRegression model from scikit-learn is created.
- The model learns to map the 768-dimensional BERT embeddings to the sentence length.

Employing Linear Regression

In short...

The linear regression takes the BERT sentence embeddings as input and learns to predict the number of words in each sentence.

Linear Regression on BERT Sentence Embeddings



Metric	Value
MSE	18.289
RMSE	4.277
MAE	3.302
R ²	0.837

Linear Regression on BERT Sentence Embeddings

Putting that into perspective

MSE

- **What it means:**
 - On average, the squared difference between the predicted and actual sentence lengths is 18.3.
- **In context:**
 - If the model predicts a sentence should be 10 words, but it's actually 14, that's a squared error of 16.
 - Most of the predictions are not far off.

Metric	Value
MSE	18.289
RMSE	4.277
MAE	3.302
R²	0.837

Linear Regression on BERT Sentence Embeddings

Putting that into perspective

RMSE

- **What it means:**
 - The typical prediction is off by about 4.3 words.
- **In context:**
 - If a sentence is 20 words, the model might predict anywhere from 16 to 24 words.
 - The model is capturing the general length, but not always the exact count.

Metric	Value
MSE	18.289
RMSE	4.277
MAE	3.302
R ²	0.837

Linear Regression on BERT Sentence Embeddings

Putting that into perspective

MAE

- **What it means:**
 - On average, predictions are off by about 3.3 words.
- **In context:**
 - For every sentence, expect your model to be within 3–4 words of the true length.

Metric	Value
MSE	18.289
RMSE	4.277
MAE	3.302
R ²	0.837

Linear Regression on BERT Sentence Embeddings

Putting that into perspective

R²

- **What it means:**
 - The model explains about 84% of the variance in sentence length.
- **In context:**
 - Strong result.
 - BERT embeddings seem to contain a lot of information about sentence length, and linear regression is able to extract most of it.
 - There's still 16% of the variation that the model can't explain.

Metric	Value
MSE	18.289
RMSE	4.277
MAE	3.302
R ²	0.837

Linear Regression on BERT Sentence Embeddings

Cross-validation

Not Cross-validated

Metric	Value
MSE	18.289
RMSE	4.277
MAE	3.302
R ²	0.837



Cross-validated
k= 5 folds

Metric	Value
MSE	17.022
RMSE	4.124
MAE	3.208
R ²	0.840

Little improvement...

Linear Regression on BERT Sentence Embeddings

Seeing it happening

Sentence:
In 1973 , Wells Fargo & Co. of San Francisco launched the Gold Account , which included free checking , a credit card , safe-deposit box and travelers checks for a \$ 3 monthly fee .

Actual Length: 36
Predicted Length: 37.14

Index	Tokens
1	In
2	1973
3	,
4	Wells
5	Fargo
6	&
7	Co.
8	of
9	San
10	Francisco
11	launched
12	the
13	Gold
14	Account
15	,
16	which
17	included
18	free
19	checking
20	,
21	a
22	credit
23	card
24	,
25	safe-deposit
26	box
27	and
28	travelers
29	checks
30	for
31	a
32	\$
33	3
34	monthly
35	fee
36	.

Linear Regression on BERT Sentence Embeddings

Seeing it happening

Sentence:

Campbell Soup , for one , is furious its Souper Combo microwave product was chastised in the premiere `` In the Dumpster '' column .

Actual Length: 25

Predicted Length: 27.50

Index	Tokens
1	Campbell
2	Soup
3	,
4	for
5	one
6	,
7	is
8	furious
9	its
10	Souper
11	Combo
12	microwave
13	product
14	was
15	chastised
16	in
17	the
18	premiere
19	``
20	In
21	the
22	Dumpster
23	''
24	column
25	.

Linear Regression on BERT Sentence Embeddings

Summarizing all that...

- Linear Regression is quite good at predicting sentence length from BERT embeddings.
- Most predictions are within 2–4 words of the actual length.
- A few sentences are much harder to predict, but these are rare.
- The model captures most of the information about sentence length present in the embeddings.
- For practical purposes, the approach is effective, but for more precise predictions, you may need a more complex model.

Probing for Verbs: Preprocessing Changes

Changes to preprocessing

- We still remove trace tokens.
- A list is created containing plain sentences.
- We identify verb POS tags to define the ground truth labels.
- This list is used to derive the binary target: *contains verb or not*.

Tag	Description	Example
VB	Base form	run
VBD	Past tense	ran
VBG	Present participle	running
VBN	Past participle	eaten, run
VBP	Present tense, non-3rd person singular	run, eat
VBZ	Present tense, 3rd person singular	runs, eats

Class Imbalance in Verb Presence

Same Embeddings, New Task

We reuse:

- The same cleaned input sentences.
- The same tokenizer and BERT model.
- The same embedding function (CLS-based).

But this time:

- The target variable is **verb presence** (binary: 0/1).
- Each sentence is labeled based on its POS tags.

Class Imbalance in Verb Presence

The pipeline



But, It's Not So Easy

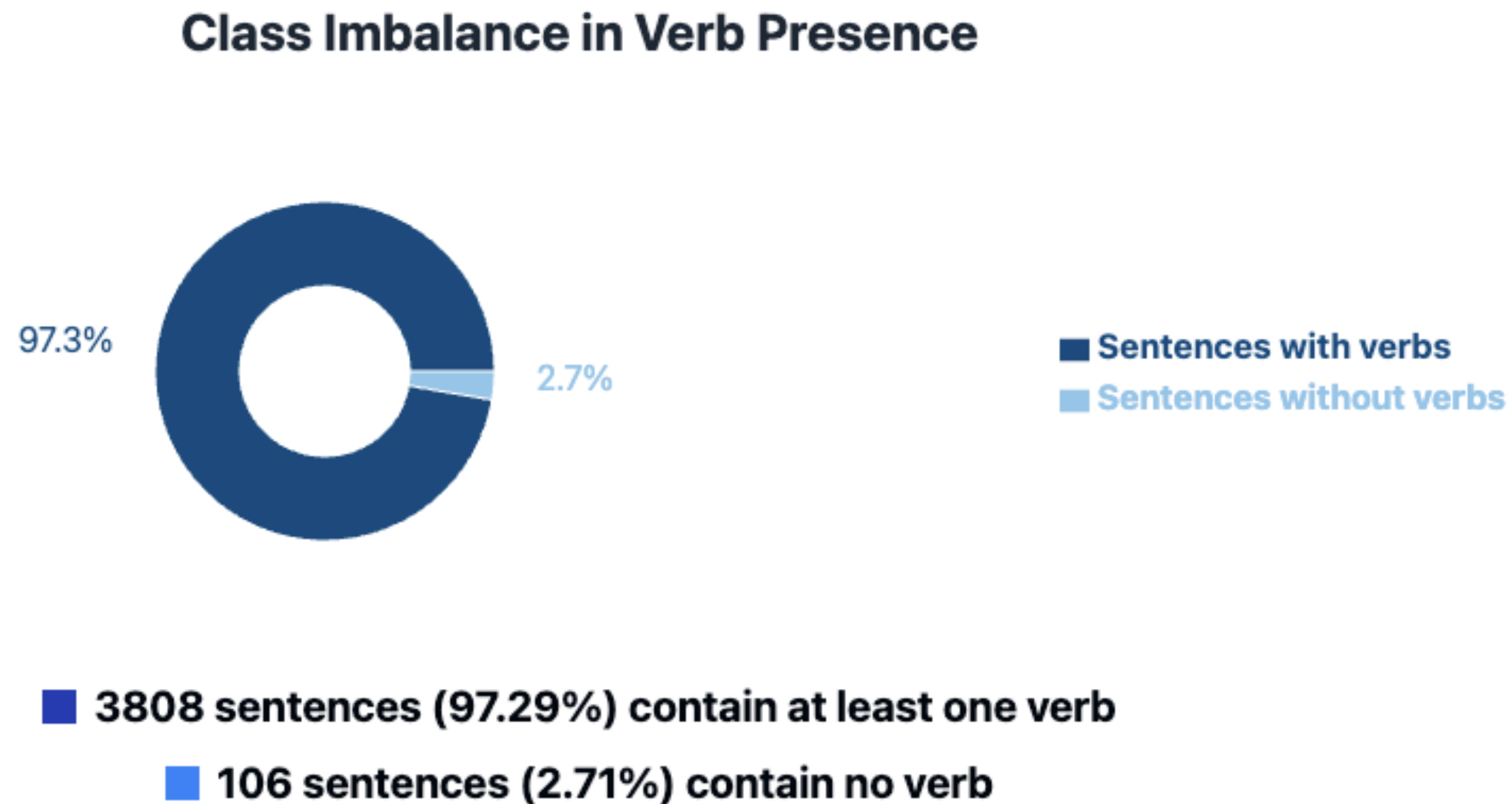
Class Imbalance in Verb Presence

- 3808 sentences (97.29%) contain at least one verb.
- 106 sentences (2.71%) contain no verb.
- No outliers removed, unlike in the sentence length task.

 ***Class imbalance: nearly all sentences contain verbs***

Class Imbalance in Verb Presence

Visualizing that...



Class Imbalance in Verb Presence

Create a Balanced Train Set (Down-sampling)

Why down-sampling?

- Keeps the **original sentences** (no synthetic data).
- Simple and interpretable.
- Guarantees a 50/50 class balance in training.
- Reduces training size quite a bit in this case.

Class Imbalance in Verb Presence

Create a Balanced Train Set (Down-sampling)

1. Stratify test set — maintain natural imbalance (97/3) for realistic evaluation
2. Split classes — isolate verb vs. no-verb examples
3. Down-sample majority class — match class sizes
4. Recombine & shuffle — build a balanced, randomized train set
5. Result: Equal # of verb/no-verb sentences in training

The down-sampling strategy keeps 85 negative examples and randomly selects 85 positive ones to create a balanced training set of 170 sentences.

Class Imbalance in Verb Presence

That's not a lot of sentences right?

- **70 sentences is very small** compared to your original training size.
 - A **94.6% reduction** in available training data.
-

Class Imbalance in Verb Presence

Results to interpret

Test Accuracy: 0.959
Test Precision (Verb Present): 1.000
Test Recall (Verb Present): 0.958
Test F1-Score (Verb Present): 0.979

Classification Report (Logistic Regression):

	precision	recall	f1-score	support
No Verb	0.40	1.00	0.57	21
Verb Present	1.00	0.96	0.98	762
accuracy			0.96	783
macro avg	0.70	0.98	0.77	783
weighted avg	0.98	0.96	0.97	783

- No Verb (Negative class, support=21):
 - Precision: 0.40**
When the model predicts "No Verb", it's correct 40% of the time.
 - Recall: 1.00**
The model finds all actual "No Verb" sentences (no false negatives).
 - F1-score: 0.57**
Harmonic mean of precision and recall; moderate due to low precision.
- Verb Present (Positive class, support=762):
 - Precision: 1.00**
When the model predicts "Verb Present", it's always correct (no false positives).
 - Recall: 0.96**
The model finds 95.8% of all actual "Verb Present" sentences.
 - F1-score: 0.98**
Very high, reflecting strong performance on this class.

Class Imbalance in Verb Presence

Results to interpret

Test Accuracy: 0.959
Test Precision (Verb Present): 1.000
Test Recall (Verb Present): 0.958
Test F1-Score (Verb Present): 0.979

Classification Report (Logistic Regression):				
	precision	recall	f1-score	support
No Verb	0.40	1.00	0.57	21
Verb Present	1.00	0.96	0.98	762
accuracy			0.96	783
macro avg	0.70	0.98	0.77	783
weighted avg	0.98	0.96	0.97	783

- After down-sampling:
The training set (X_train_balanced, y_train_balanced) contains 170 sentences (85 with verbs, 85 without verbs).
This is the data used to train the model.
- Test set:
The test set (X_test, y_test) is not down-sampled.
It keeps the original class distribution and size, which in your output is 783 sentences.

•

Entropy of BERT embeddings and sentence length prediction

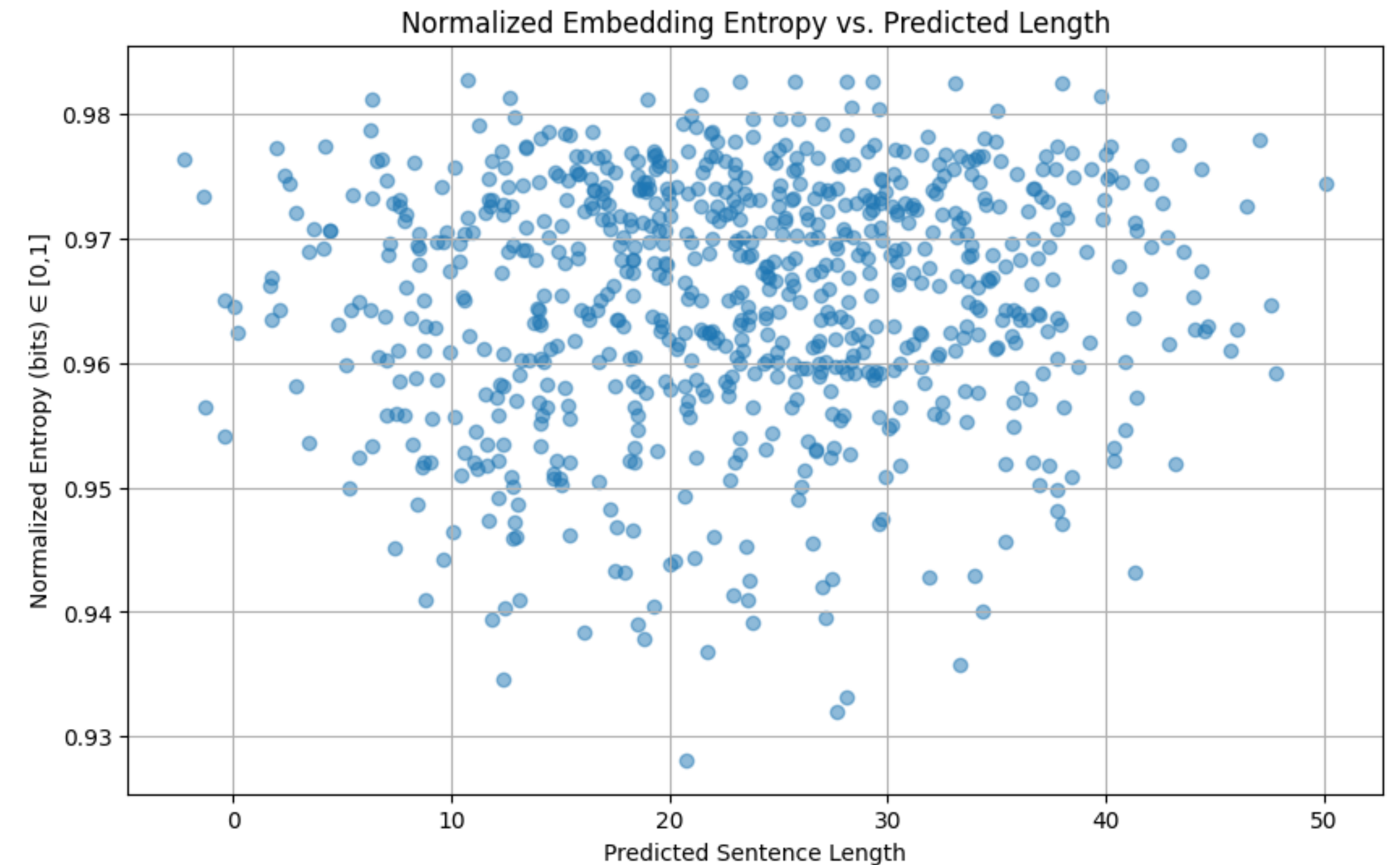
Theory

If BERT embeddings for sentences have low entropy, sentence length should be easier to predict.

Entropy of BERT embeddings and sentence length prediction

Results

Norm Entropy Min: 0.928
Norm Entropy Max: 0.983
Norm Entropy Mean: 0.965



Embeddings are high-entropy

Entropy of BERT embeddings and sentence length prediction

Conclusion

- **High entropy** in BERT embeddings does **not** prevent accurate prediction of sentence length.
- Low entropy is not necessary for easy prediction in this case.

Past–Present Probing in BERT

Walkthrough of probing for verb tense...

- **Goal:** Use BERT's [CLS] token embedding to train a binary classifier that predicts whether a sentence is in past or present tense.
- Directly testing whether BERT carries tense information.

Past–Present Probing in BERT

Tag Definition

- Define POS tag sets:
 - `present_tags = {'VB', 'VBP', 'VBZ'}`.
 - `past_tags = {'VBD', 'VBN'}`.
- Collect all sentences with at least one verb tag.

Past–Present Probing in BERT

Label Assignment & Filtering

- **Collect tense tags** in each POS-tagged sentence.
- **Filter for exactly one tense** to guarantee a clean binary label:
 - **Drop mixed-tense** sentences containing both past and present verbs.
 - **Drop verbless** sentences (no tense information).
- **Result:** every example is unambiguously “past” or “present”.

Past–Present Probing in BERT

Raw data to work with

- Present tense sentences: 1728
- Past tense sentences: 1434
- Balance this, why?
 - The model may overpredict the dominant class, hurting minority class performance.
 - High accuracy can mask poor performance on the underrepresented class.
 - Balanced data helps the model learn useful features for all classes.
 - Imbalance leads to suboptimal learning.

BERT Embeddings

Same Embeddings, New Task

Reuse:

- The same cleaned input sentences.
- The same tokenizer and BERT model.
- The same embedding function (CLS-based).

But this time:

- The target variable is **the verb tense of a sentence** (binary: 0/1).
- Two classes:
 - "past" (e.g., containing verbs like *went*, *was*, *had*)
 - "present" (e.g., containing verbs like *go*, *is*, *has*)

Logistic Regression

Input

BERT [CLS] token embeddings

- One 768-dimensional vector per sentence
- Extracted from the variable: *X_balanced_sentence_features*

Prediction Target

The output is the predicted label for each sentence:

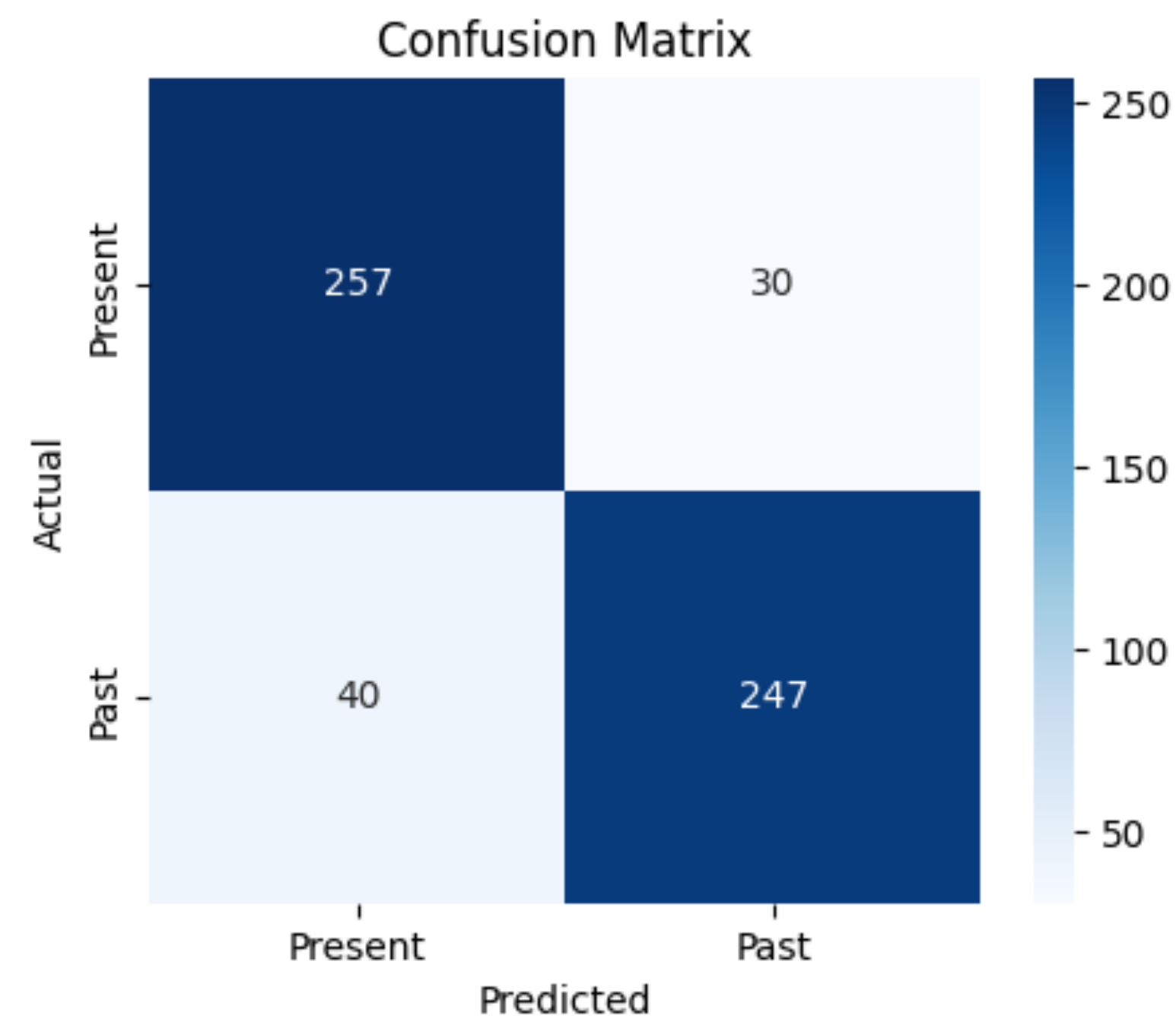
- *0* for present tense
- *1* for past tense

This is stored in *y_pred*, and the true labels are in *y_balanced* (split into *y_train* and *y_test* for training and evaluation).

Logistic Regression Results

Evaluation Metrics

	precision	recall	f1-score	support
Present	0.87	0.90	0.88	287
Past	0.89	0.86	0.88	287
accuracy			0.88	574
macro avg	0.88	0.88	0.88	574
weighted avg	0.88	0.88	0.88	574



Logistic Regression Results

Interpreting...

Overall Accuracy: 88%

- The model correctly identifies the dominant tense (past vs. present) in 88% of cases.

Precision & Recall Breakdown:

- For present tense, the precision is 0.87, meaning that when the model predicts “present”, it’s correct 87% of the time.
- The recall is 0.90, indicating that 90% of all true “present” sentences are successfully identified.
- For past tense, the precision is 0.89, so predictions labeled as “past” are accurate 89% of the time.
- The recall is 0.86, meaning the model captures 86% of actual past-tense cases.

Logistic Regression Results

Interpreting...

Balanced Performance:

- F1-scores are equal at 0.88 for both present and past classes.
 - This shows that the model handles both categories with equal effectiveness, indicating that the class balancing step was successful and that the model does not favor one tense over the other.

Confusion Matrix Insight:

- 30 present-tense sentences were misclassified as past, and 40 past-tense sentences were misclassified as present.
 - These errors are relatively few and evenly distributed, which suggests the model is making consistent and unbiased predictions.

Logistic Regression Results

Summarizing...

The model is reliably distinguishing between present and past tense sentences, achieving balanced precision and recall across both categories. With an overall accuracy of 88%, and minimal class confusion, we can conclude that tense-related features are meaningfully captured by the model's representations.

The Goal and Big Picture

Objective

Understand why a simple Linear Regression model can predict sentence length well using BERT embeddings.

The Goal and Big Picture

Hypothesis

Although BERT embeddings are high-dimensional and designed for capturing semantics, they appear to encode sentence length in a structured and linearly separable manner, which allows a simple Linear Regression model to predict sentence length with high accuracy.

The Goal and Big Picture

Approach

Use hierarchical clustering to explore the internal structure of the sentence embeddings, and visualize the results with a dendrogram to examine whether the embeddings naturally group sentences by length.

What is the input?

- Use a NumPy array called *X_sentence_features*, with shape **(3868, 768)**.
- Each row represents a **BERT [CLS] embedding**, summarizing the meaning and structure of one sentence.
- These embeddings were extracted from the **cleaned and filtered sentence list**, *filtered_cleaned_sentences*.
- The underlying assumption is that semantically or structurally similar sentences will produce similar embeddings, allowing comparisons in embedding space.

Clustering with Ward's Method

Understanding Hierarchical Clustering

- Apply a bottom-up approach to clustering.
- Each sentence embedding begins as its own cluster.
- The algorithm iteratively merges the two closest clusters based on a chosen distance metric.
- This process continues until all embeddings form a single cluster hierarchy.

Clustering: Ward's method

Why Use Ward's Method?

- Ward's method selects merges that minimize the increase in total within-cluster variance.
- This results in compact clusters which are especially suitable for high-dimensional data such as BERT embeddings.
- It helps reveal natural structure in the data that simpler distance-based methods may miss.

Interpreting the Linked Matrix

Each row in this matrix logs a single merge step and contains four values:

1. Index of the first cluster being merged.
2. Index of the second cluster being merged.
3. Distance between the clusters, based on Ward's variance criterion.
4. Size of the new cluster formed by the merge (i.e., total number of points it contains).

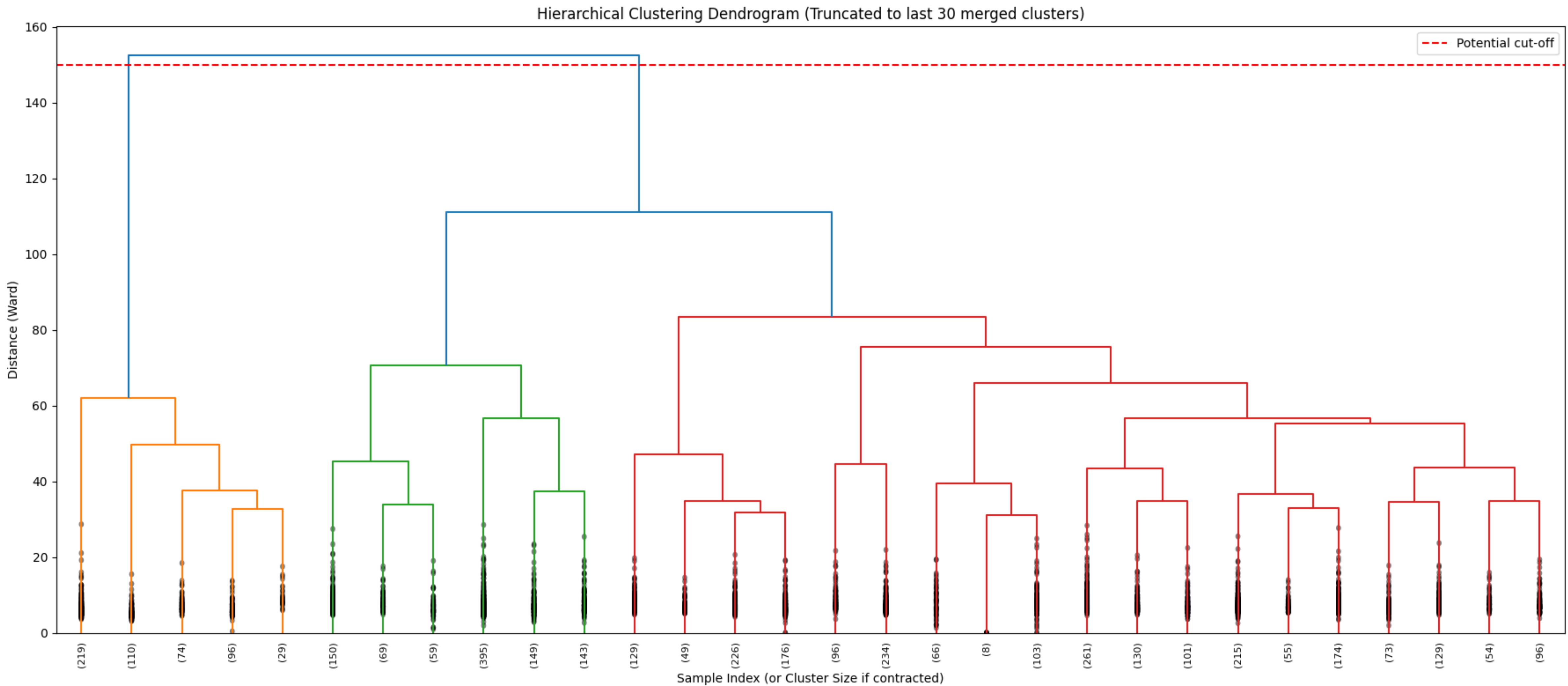
First 5 rows of the linked matrix:

```
[[5.780e+02 6.180e+02 0.000e+00 2.000e+00]
 [3.289e+03 3.793e+03 0.000e+00 2.000e+00]
 [2.210e+03 2.212e+03 0.000e+00 2.000e+00]
 [2.794e+03 2.803e+03 0.000e+00 2.000e+00]
 [9.870e+02 1.025e+03 0.000e+00 2.000e+00]]
```

Shape of linked matrix: (3867, 4)

Each row represents a merge operation with format:
[index1, index2, distance, cluster_size]

Dendrogram



Reading the Dendrogram

What does the dendrogram represent?

- It visualizes how sentence embeddings are grouped hierarchically based on similarity.
- Each merge in the tree represents two clusters being joined.
- The height of the merge indicates how dissimilar those clusters were.

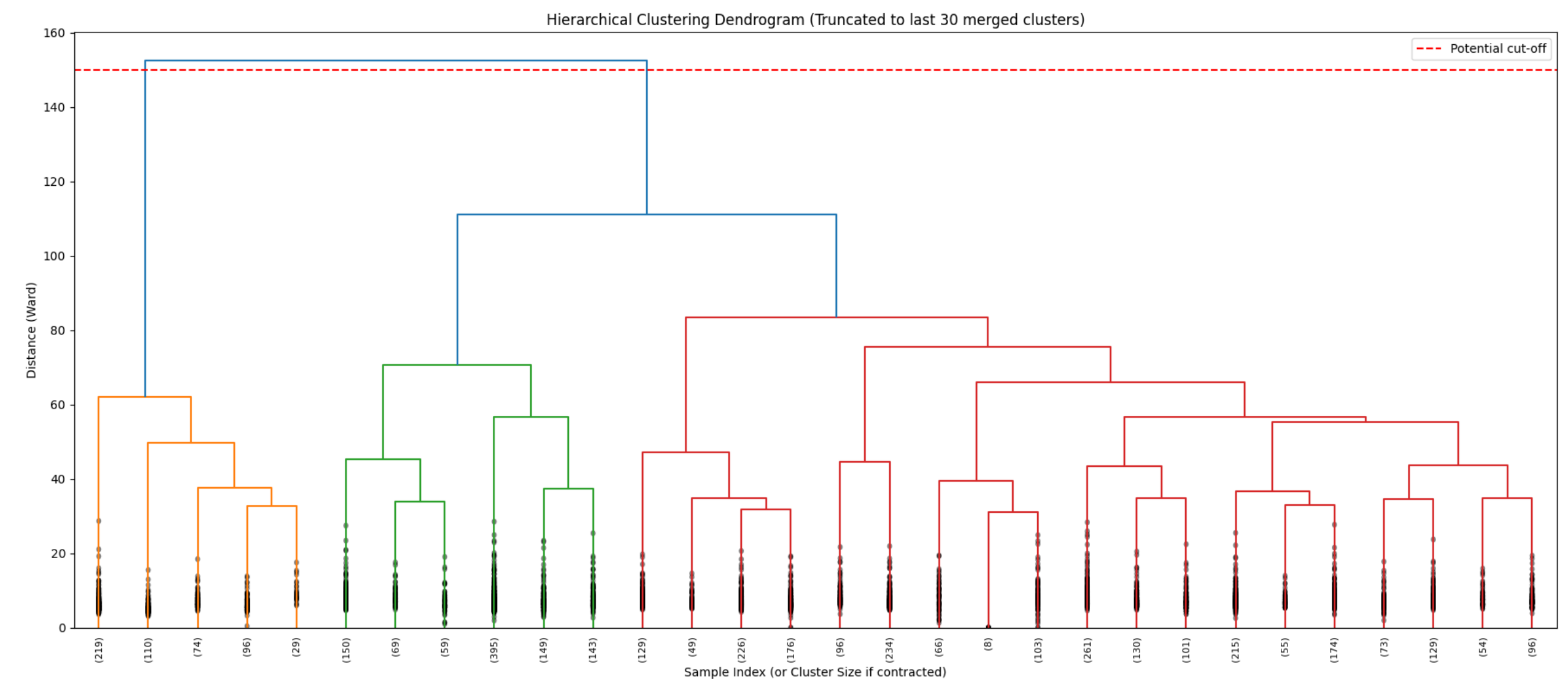
Truncated to 30 clusters

- Plotting all 3868 would be unreadable.
- Showing only the top merges helps us focus on the macro structure, namely the largest and most meaningful groupings.

Reading the Dendrogram

Understanding the axes and colors:

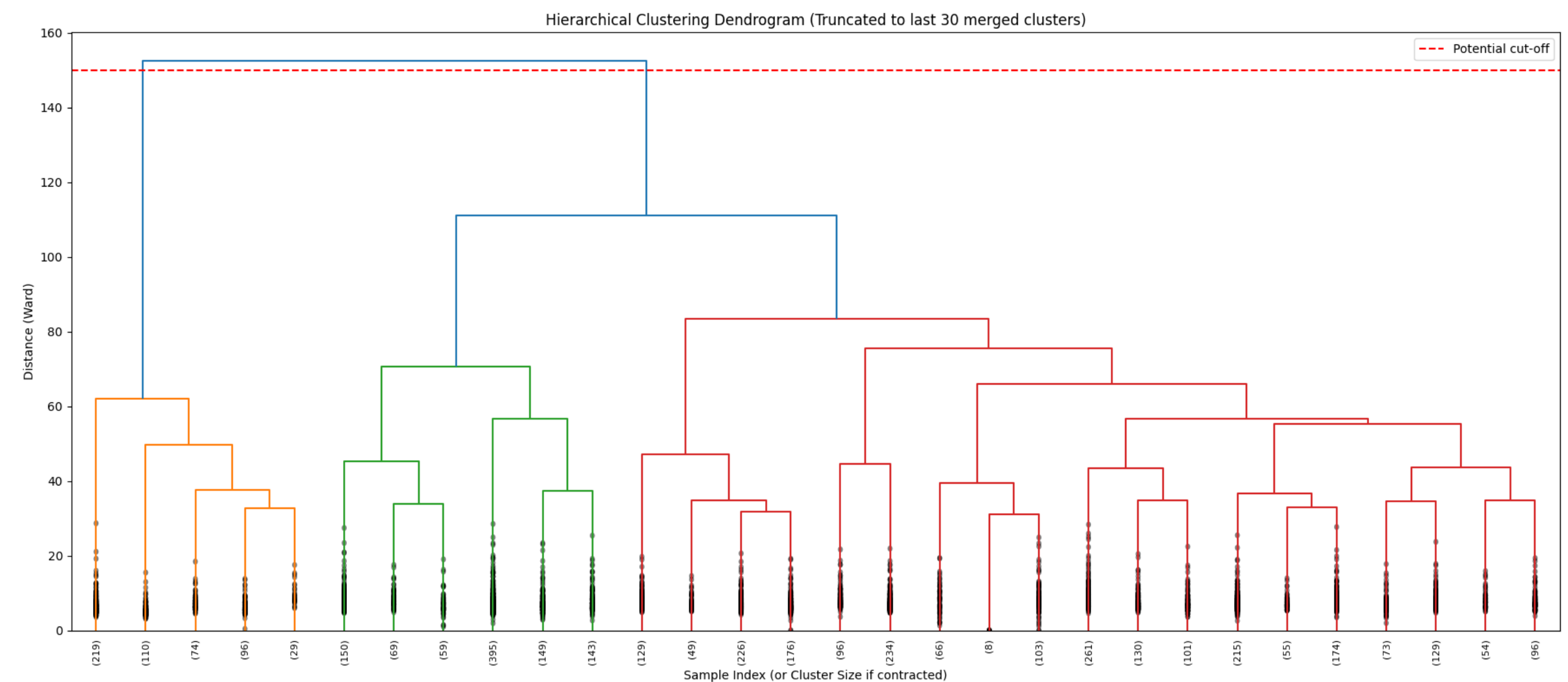
- Y-axis (Distance): Shows the dissimilarity between merged clusters (higher = more different).
- X-axis: Represents sample indices or the size of contracted clusters.
- Branch color: Used only to visually distinguish clusters.



Reading the Dendrogram

Structure and Meaning

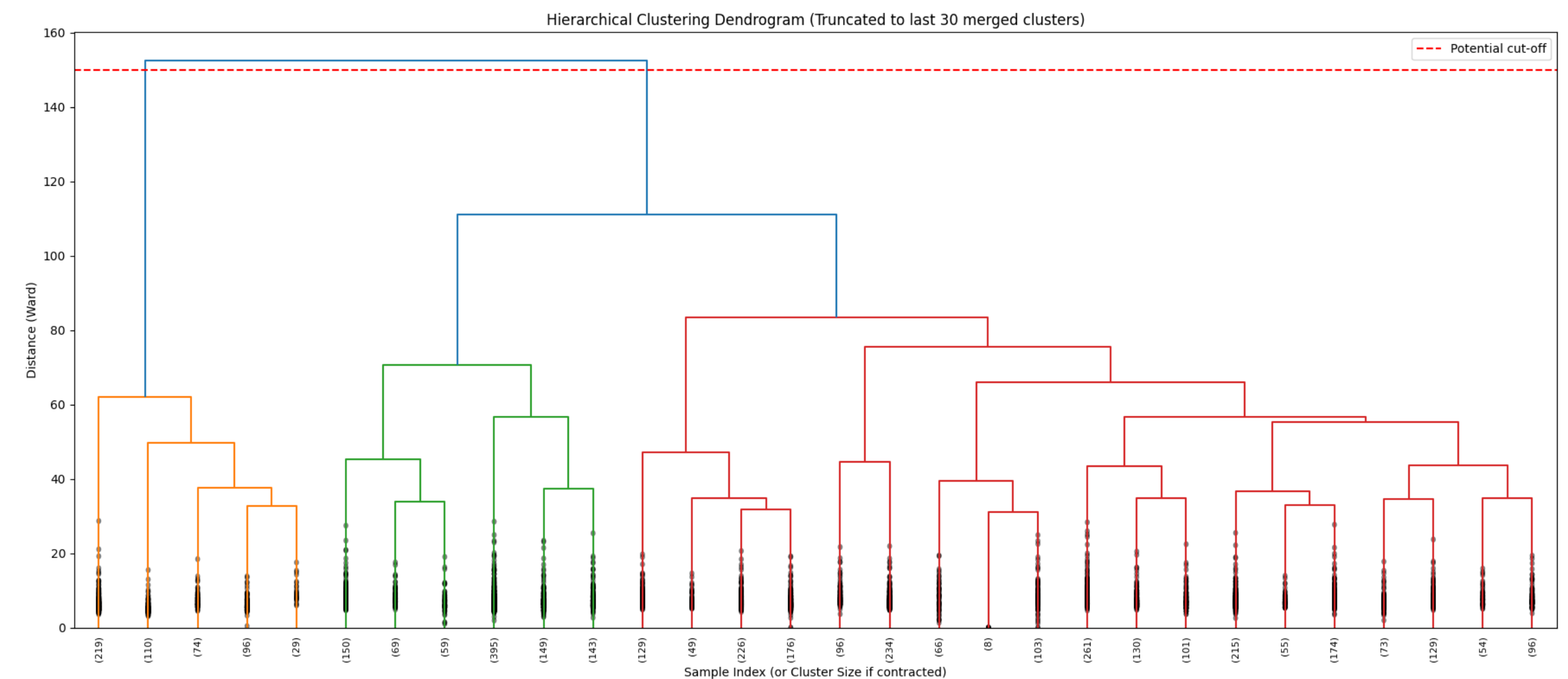
- Horizontal lines represent merge operations between clusters.
- The height of each horizontal line reflects how dissimilar the two merged clusters were.
- Vertical lines represent clusters before they were merged.
- The taller the vertical line, the more distinct that cluster was prior to merging.



Reading the Dendrogram

Red dashed line (cut line):

- If you draw a horizontal line across the dendrogram, every branch it crosses becomes a separate cluster.
- Moving the line up/down changes how fine-grained the clusters are.



What the Dendrogram Tells Us

- If sentences with similar lengths consistently fall within the same branches, it suggests that BERT embeddings organize sentences in a way that reflects length.
 - Which would support the idea that a simple linear model can exploit this structure.

So, if clusters loosely group by sentence length → Linear regression can more easily find a way to separate them.

But does it?

- I've tried different clustering cut-offs

Mean Sentence Length Per Cluster

cluster_label	
1	27.351598
2	20.409091
3	27.316583
4	25.833333
5	20.539062
6	26.845570
7	25.438356
8	23.627907
9	16.838137
10	25.489583
11	25.286325
12	14.971751
13	24.459770
14	20.051948
15	24.630631
16	23.603960
17	28.746667

Mean Sentence Length Per Cluster

cluster_label	
1	27.351598
2	24.857605
3	23.395683
4	26.247453
5	18.348276
6	25.345455
7	14.971751
8	24.093168

Mean Sentence Length Per Cluster

cluster_label	
1	25.892045
2	23.120958

I interpret this as: no distinct clustering on sentence length